Vision-Based Traffic Signal Automation and Parking System with Emergency Vehicle Prioritization

***Abstract*—The rapid rise in urban development has created traffic congestion, which can delay emergency vehicles such as ambulances and fire trucks. This can sometimes become life-threatening for many individuals. This paper introduces a vision-based traffic signal and parking automation system that integrates transfer learning for object detection, a microprocessor, sensors, and actuators for real-time implementation. To train this model, a custom dataset consisting of images of ambulances, fire trucks, and normal vehicles was created. Using this custom dataset, a model was trained and later deployed on a Raspberry Pi for lightweight inference. The system dynamically alters traffic signals by prioritizing emergency vehicles and operates normally in their absence. Moreover, the system includes an integrated smart parking subsystem that detects parking availability using sensors and wireless communication with a web interface. Evaluation in a simulated environment shows that the system achieves over 90% accuracy with very little response delay. The system is scalable, adaptable for smart cities, and cost-effective. Future improvements are aimed at expanding the dataset, testing in real-world environments, and integrating advanced edge AI accelerators.**

***Index Terms*—Traffic automation, Raspberry Pi, YOLOv9, Deep learning, Smart parking, Emergency vehicle detection, Transfer learning**

I. Introduction

In many cities, including Dhaka, the rapid rise in urban development and the growing number of vehicles on the road have made the modernization of the traffic light system a necessity. Emergency vehicles such as ambulances and fire trucks often face significant delays due to congestion and poor traffic management. These delays can have life-threatening consequences. Due to structural inefficiencies, an ambulance on average takes 85 minutes (ranging from 22 to 150 minutes) to reach the hospital. It takes about 102 minutes during the daytime, and at night, around 45 minutes. Of 734 patients, 336 (45.8%) face traffic congestion problems [1]. Similarly, during fire emergencies, firefighters are often unable to reach their destination promptly. Only in 50% of cases were they able to reach the incident location within 8 minutes [2]. Efficiently identifying and prioritizing emergency vehicles in real time is thus crucial to optimizing emergency response time.

This research presents an innovative approach that utilizes machine learning and microprocessor-based systems for real-time emergency vehicle detection and response automation. Deep Learning and Artificial Intelligence have transformed the field of computer vision by enabling systems that process visual data with high accuracy [3]. YOLO (You Only Look Once) is a widely used object detection algorithm known for its real-time performance and high accuracy [4]. Its efficiency makes it suitable for applications such as traffic monitoring and emergency vehicle detection. Recent versions of YOLO, like YOLOv5, have improved detection accuracy, especially for small objects, through advanced training techniques such as data augmentation [5].

In this study, we applied the YOLO object detection algorithm to identify emergency vehicles such as ambulances and fire trucks from live camera feeds. The model is trained with 360 labeled images containing various types of vehicles, including normal vehicles, fire trucks, and ambulances. The dataset was annotated using Roboflow and further processed to enhance model accuracy. The training was performed using Kaggle's computing resources, and the trained model was deployed on a Raspberry Pi 4 for a real-time application.

The primary goal of this project is to integrate trained YOLO models with a microprocessor-based system to automate emergency response mechanisms. The Raspberry Pi, along with a webcam, continuously captures live footage and processes it using the trained model. Upon detecting an emergency vehicle, the system can send API requests to control traffic signals, alert nearby vehicles, or trigger predefined response actions. This automation aims to reduce human intervention, enhance traffic efficiency, and ensure that emergency vehicles reach their destinations with minimal delay.

The proposed system's effectiveness is evaluated based on detection accuracy, response time, and computational efficiency. Results show that the system reliably and confidently identifies emergency vehicles in real time, enabling timely interventions. Moreover, the use of Raspberry Pi ensures cost-effectiveness and scalability, making the solution practical for smart city implementation.

II. Literature Review

This section briefly discusses the prior studies on emergency vehicle detection and traffic management using computer vision, deep learning, and real-time data analysis.

Dhaka City faces severe traffic congestion due to rapid urbanization and an increase in the number of vehicles. One major concern is the slow movement of emergency vehicles due to heavy traffic, and very few studies have focused on solving this issue. For example, Razalli et al. [6] aimed to develop a system to automatically detect and classify emergency vehicles like ambulances, fire trucks, and police cars from traffic recorded video. Their motivation was that emergency vehicles get stuck in traffic, causing delays. An automatic system might solve the situation. They designed a

computer vision-based algorithm that used Color Segmentation models like HSV and RGB to detect sirens' lights. They used a Support Vector Machine (SVM) to distinguish whether a vehicle is an emergency vehicle or not. They achieved 97.3% accuracy with low processing time. But the system heavily relies on the siren's light color and may misclassify if the background vehicles contain the same colors.

Shinde et al. [7] aimed to reduce traffic congestion and improve traffic speed at the intersection. They adjusted the light of traffic based on density and waiting times at the points of intersection. So they used YOLO for vehicle detection and embedded hardware like a Raspberry Pi. Using a camera, they calculated the density according to their threshold and controlled the light. But, they had low accuracy due to low-resolution objects, hardware optimization, etc. Kavya et al. [8] also developed a similar system using infrared sensors to measure the levels of congestion on the road. George et al. [9] improved traffic management and reduced congestion using IoT for real-time monitoring. They built a smart traffic light system that adjusts the signal based on vehicle density and waiting time rather than a fixed time. They had a neat setup, but they relied on a low-processing-power Arduino. Also, there was no emergency vehicle priority. Transfer learning to detect emergency vehicles from CCTV footage was introduced in [10], [11], and [12]. They achieved a high accuracy in vehicle classification as an emergency or normal vehicle. Thus, these papers present the usability of deep learning in a real-time traffic monitoring system. However, the systems also had some drawbacks, such as they might lose accuracy if visibility is low or in conditions of heavy traffic. These remaining challenges need further optimization.

An adaptive traffic control system that dynamically allocates the duration of the green light using image processing and edge detection techniques was proposed by Meng et al. [13]. It compared five edge detection methods—Roberts, Sobel, Log, Canny, and Active Contour—to determine the most efficient approach, and the Canny edge detection technique was found to be the most effective in accurately detecting traffic density while maintaining low processing time. Geethapriya et al. [14] aimed to provide a comprehensive review of real-time object detection methods using the YOLO algorithm and Convolutional Neural Networks (CNN). Its goal was to analyze the evolution of YOLO models, compare them with other detection techniques such as R-CNN and SSD, and highlight their applicability in real-time domains. It achieved this by summarizing YOLO's evolution, highlighting its speed, efficiency, and applications in areas like autonomous driving and surveillance. However, its limitations include localization errors, difficulty with small/overlapping objects, hardware dependency, and the fact that it is a survey rather than a novel contribution.

Chockalingam et al. [15] designed a double-verification system for the ambulance: using microphones, it detected the siren sound, while a Raspberry Pi camera confirmed the presence of the ambulance. Once verified, the system immediately switched the traffic light to green in the ambulance's direction and sent notifications to police and drivers. However, the system's limitations include possible inaccuracies in noisy environments, reduced image detection in poor lighting/weather conditions, and limited real-world scalability. Mecocci et al. [16] combined audio (siren detection using MFCC features) and video (ambulance recognition using customized YOLOv8 models) to ensure timely green signals for ambulances, particularly in challenging cases like one-way roads or temporary roadworks. It introduced a part-based model with Lookout Stations that provided timely green signals, working effectively even in noisy, nighttime, or adverse weather conditions. Its limitations include dependency on sensor quality and placement, the need for region-specific training data, and challenges in scaling to complex urban networks.

Although some studies have contributed significantly to emergency vehicle detection and traffic signal control, most solutions focus on either detection or traffic optimization rather than fully integrating both. Many existing systems achieve high accuracy in emergency vehicle classification but fail to implement real-time traffic light adjustments. This paper bridges this gap by developing a system that identifies emergency vehicles and optimizes traffic flow dynamically, ensuring their uninterrupted passage through congested areas. Moreover, a web interface is developed to continuously monitor and control the system.

III. Methodology

The proposed system follows a modular and step-by-step pipeline to manage traffic control and parking based on real-time vehicle detection. The methodology is divided into 5 key phases: (a) data preparation and model training, (b) real-time video feed and interface, (c) emergency vehicle and density detection logic, (d) traffic signal control, and (e) smart parking system and hardware actuation.

A. Data Preparation and Model Training

A custom dataset had been generated using miniature plastic vehicles to simulate real-world traffic. Images were captured under various lighting and angle conditions and labeled into three classes: *ambulance*, *fire truck*, and *normal vehicle*. Some example images from the dataset are illustrated in Fig. 1. The dataset can be found here: https://github.com/NajmusSabir99/Emergency-Vehicle-and-Density-Detection. To train the model on our dataset, transfer learning was applied using the YOLOv9 pre-trained object detection model.

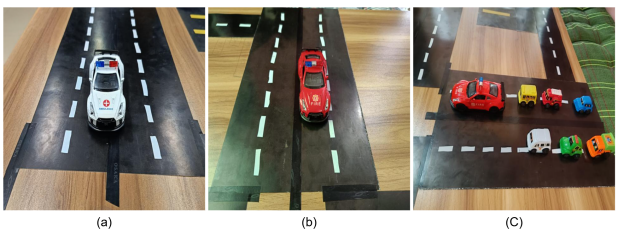

Fig. 1. Dataset images: (a) Ambulance, (b) Fire Truck, (c) Normal Vehicles

B. Real-Time Video Feed and Inference

A camera module connected to a Raspberry Pi captures real-time video from the simulated environment. The trained model is deployed on the Raspberry Pi to process each frame and classify the detected vehicles accordingly.

C. Emergency Vehicle and Density Detection Logic

- If an emergency vehicle (*ambulance* or *fire truck*) is detected for at least three consecutive frames, that lane is prioritized, and the corresponding traffic signal is switched to green.
- If no emergency vehicle is detected, the system calculates the vehicle density per lane and assigns green lights based on the lane with the highest density.

D. Traffic Signal Control

The Raspberry Pi utilizes its GPIO pins to control the LED indicators representing the red, yellow, and green traffic lights for each lane. Signal logic is dynamically updated in real-time based on the detection outcomes.

E. Smart Parking System

The system also includes a smart parking module, which monitors and displays the status of each parking slot based on sensor inputs. The availability of slots is determined using IR sensors, and the data is processed using a Raspberry Pi. This processed information is used to indicate the occupancy status of each slot. The hardware layout for this module is illustrated in Fig. 2. The modules communicate through HTTP requests using a Flask-based server. Based on classification and analysis, the detection script sends an HTTP signal to the traffic signal controller and the parking interface.

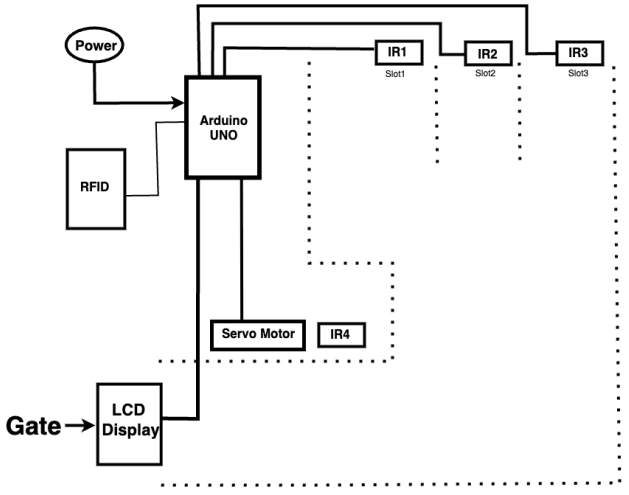

Fig. 2. Circuit diagram of the Smart Parking System module.

Fig. 3 illustrates the complete workflow of the proposed system, from video feed capture to decision-making and hardware response.

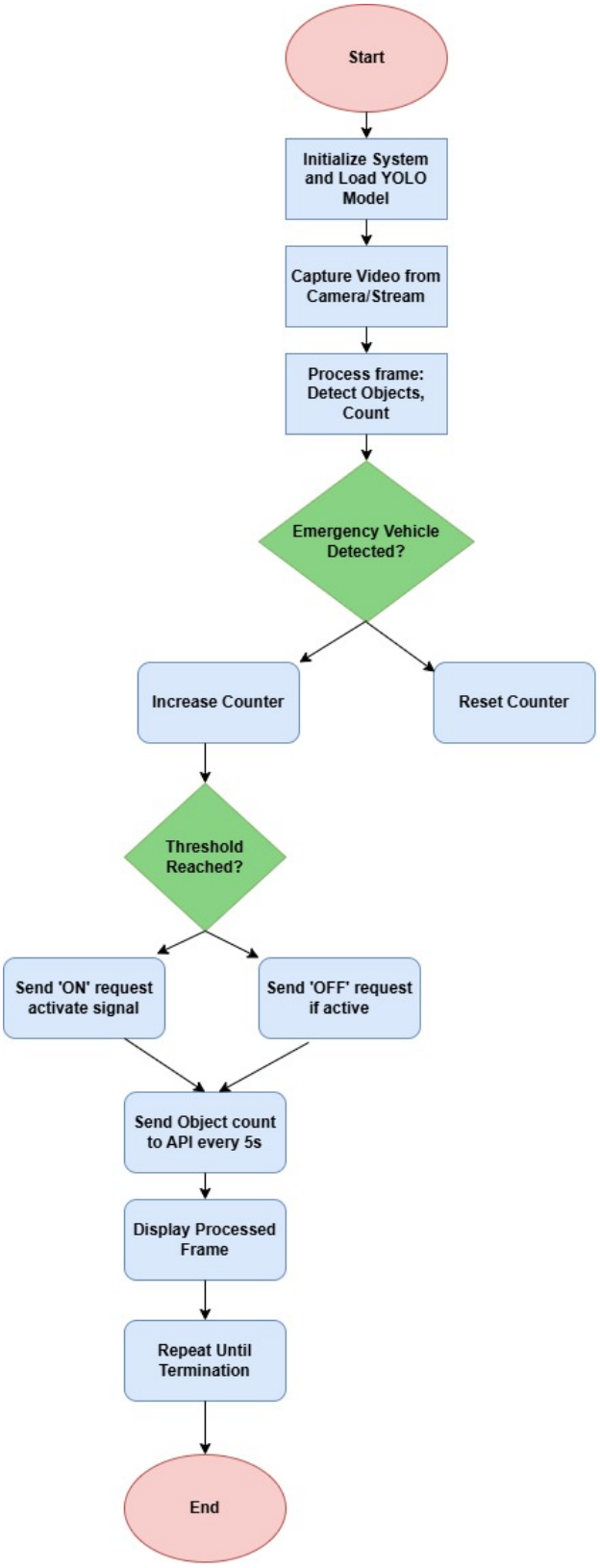

Fig. 3. Flowchart of the proposed traffic automation and parking system.

IV. Design and Development

The proposed system was developed using a combination of hardware components, deep learning models, and web technologies to simulate a real-time, vision-based traffic signal automation and parking management system. The system visual overview is shown in Fig. 4. This visual representation includes the Raspberry Pi unit, camera input, detection module, LED signal outputs, and parking slot status monitor. It provides an intuitive understanding of how the system operates.

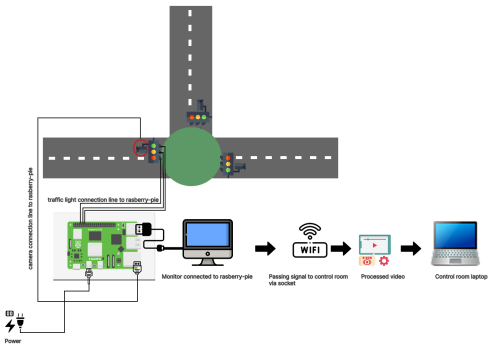

Fig. 4. System Overview: Raspberry Pi-powered vision-based traffic control and parking detection simulation.

The following component are involved in the system development:

A. Hardware Components

Raspberry Pi: This microprocessor serves as the central controller, running the trained model, and handling GPIO outputs to control traffic LEDs. Its built-in Wi-Fi and sufficient RAM make it capable of handling lightweight inference and server communication tasks.

Camera Module: A generic USB webcam is used for the real-time video feed capture. It is mounted in such a way that it can observe multiple lanes and parking areas simultaneously.

Traffic Signal Simulation: The traffic system was simulated using LEDs connected to the Raspberry Pi's GPIO pins. Each lane was assigned a set of red, yellow, and green LEDs representing actual traffic lights. Our simulated environment is presentend in Fig. 5

Breadboard and Resistors: Used to safely interface LEDs with the GPIO pins. 330-Ohm resistors were used to prevent overcurrent damage.

Miniature Vehicles: Plastic models of normal cars, ambulances, and fire trucks were used to simulate real-world traffic in the testing environment.

B. Software Development

Training Our Model: The YOLOv9 model was trained using a custom dataset labeled via Roboflow. A total of 360 images were annotated across three classes: normal vehicles, fire trucks, and ambulances. The training was conducted on Kaggle using GPU acceleration. After training, the model was exported in ONNX format and optimized for Raspberry Pi.

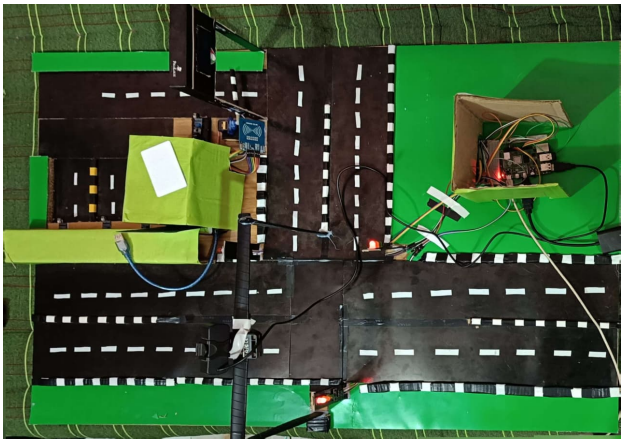

Fig. 5. Simulated Environment

Real-Time Detection and Signal Logic: A Python script using OpenCV and Ultralytics libraries processes video frames in real-time. The script:

- Runs inference on every 3rd frame
- Applies a confidence threshold of 0.65
- Maintains detection counters to reduce false triggers
- Sends GPIO signals to switch LED lights based on the detected vehicle class and density

Smart Parking System: A separate Python module tracks available parking slots using object detection. Results are sent to a local Flask-based server that renders the data on a web page in real time. Users can view the current parking status through the visual interface.

C. Integration and Communication

The detection module, traffic control module, and parking system operate as separate Python scripts. They communicate via RESTful APIs exposed by the Flask server. The detection script sends real-time detection events and density values to the traffic controller, which updates signal timings accordingly. Similarly, the parking module sends occupancy status to the web server.

D. Challenges and Optimization

- **Model Size:** YOLOv9 is large and difficult to run natively on Raspberry Pi. Quantization and ONNX export helped mitigate this.
- **Frame Drop:** Real-time inference required balancing performance by skipping frames.
- **False Positives:** Multiple-frame confirmation and a confidence threshold reduced noisy predictions.

This design ensures modularity, cost-effectiveness, and real-time adaptability, making it suitable for deployment in developing urban infrastructures.

V. Evaluation and Result Analysis

To assess the effectiveness and reliability of our traffic signal automation system, the performance of our model and system response were thoroughly evaluated.

A. Real-Time Video Feed and Inference

A USB camera connected to a Raspberry Pi captures real-time video from the simulated environment. The inference speed of the deployed model fluctuates between 10 to 15 frames per second (FPS), depending on lighting and system load. To optimize performance, the system analyzes every third frame, balancing computational efficiency with detection accuracy.

Our trained model is deployed on the Raspberry Pi using the ONNX format for lightweight inference. A confidence threshold of 0.65 is used to minimize false positives. When an ambulance or fire truck is detected in three consecutive analyzed frames, the system initiates an API request to activate the emergency response mechanism—such as changing traffic signals. If emergency detection is not sustained, a signal is sent to deactivate the response.

To ensure reliable object detection and maintain system integrity, an object count feature monitors vehicle movement in real time and periodically transmits this data via API for analytical logging and visualization. A Flask-based server manages communication between modules, handling real-time streaming and API requests to and from the detection system.

B. Model Training Evaluation

The model was trained on a custom dataset of 360 annotated images categorized into three classes: normal vehicle, ambulance, and fire truck. The dataset was processed and annotated using Roboflow. Various training augmentations, such as mosaic, flipping, and brightness adjustment, were applied to enhance robustness under different conditions.

C. Annotation Heatmap

Fig. 6 illustrates the annotation heatmap generated from the labeled dataset. It shows that vehicle annotations are well distributed across the frame space, ensuring that the model generalizes well for detecting objects at various positions.

D. Performance Metrics

The model achieved strong performance in terms of precision, recall, and overall accuracy, as shown in Fig. 7. These metrics indicate the model's ability to detect emergency vehicles accurately while minimizing false positives.

E. Class Distribution and Confidence

A histogram depicting the number of detections per class is provided in Fig. 8. It confirms a balanced distribution of annotated classes and shows confidence levels of predictions across the dataset.

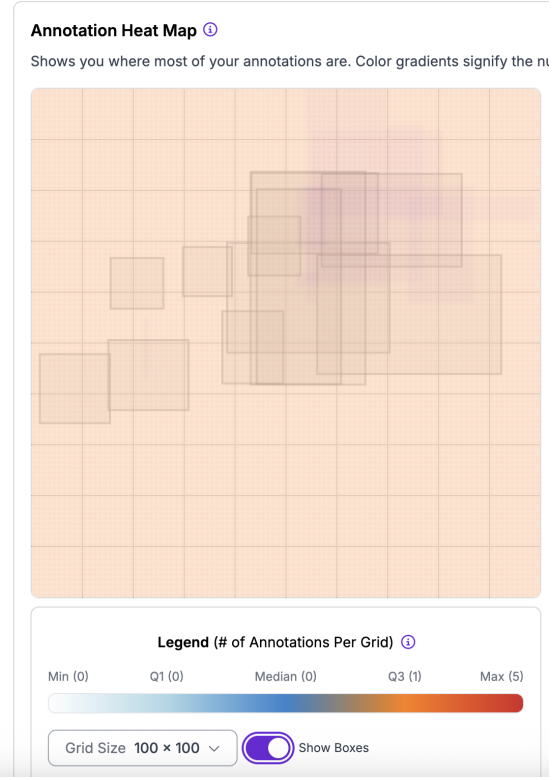

Fig. 6. Annotation Heatmap showing annotation density across image regions.

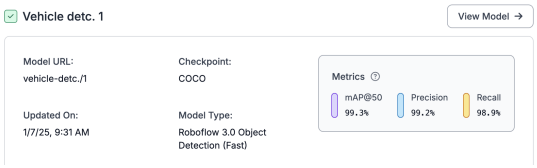

Fig. 7. Accuracy, Precision, and Recall metrics after model training.

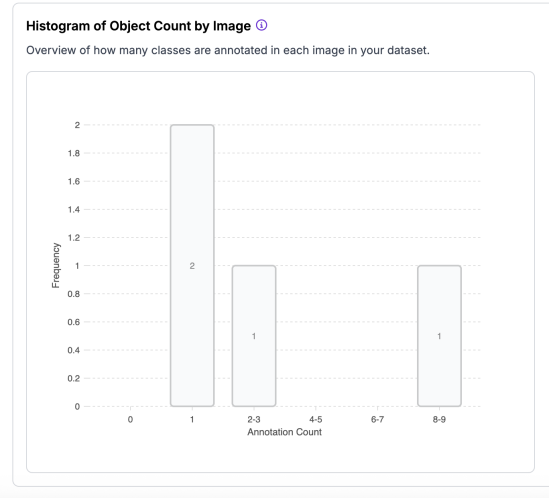

Fig. 8. Class distribution and confidence histogram of the detection model.

VI. System Demonstration

To validate the practical performance of the system, a real-time screenshot was captured during the detection of vehicles using our model (see Fig. 9). This demonstration highlights how the system accurately identifies ambulances, fire trucks, and normal vehicles by drawing bounding boxes and displaying class labels over them.

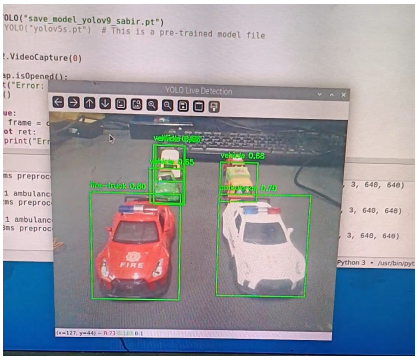

Fig. 9. Screenshot showing YOLOv9-based real-time emergency vehicle detection.

A. System Performance in Real-Time

In the simulation environment, the system was able to:

- Detect emergency vehicles with over 90% accuracy
- Reduce average signal delay for emergency vehicles by dynamically altering lights
- Achieve real-time inference at approximately 5–7 FPS on Raspberry Pi 4 by analyzing every third frame

The modular architecture allowed smooth API communication and fast GPIO-based LED signaling. Emergency response time was significantly optimized, and false activation was minimized through multi-frame detection logic.

VII. Conclusion and Future Work

In this study, we have developed and demonstrated a vision-based traffic signal automation and smart parking system using Raspberry Pi and the YOLOv9 deep learning model. By combining real-time object detection with a custom-trained dataset, the system successfully identifies and prioritizes emergency vehicles such as ambulances and fire trucks. The integration of density-based signal control and an API-driven smart parking module adds to the system's flexibility and functionality.

The proposed system was evaluated in a controlled simulation environment, where it showed reliable detection accuracy and effective traffic signal management. Key innovations include consecutive-frame emergency detection logic, modular design via Flask APIs, and real-time visualization of parking availability. The use of a Raspberry Pi ensures low-cost deployment and scalability for urban environments like Dhaka.

However, some limitations remain. The detection system is trained on a relatively small dataset and tested within a simulated environment. Performance in real-world, high-traffic, and low-light conditions requires further validation. Additionally, latency can be reduced further to support higher frame rates for detection.

Future work includes:

- Expanding the dataset with real-world images and scenarios, including nighttime and adverse weather conditions.
- Integrating GPS and traffic APIs to enhance routing and prediction capabilities.
- Deploying and testing the system in a real-world street intersection for live feedback and adaptation.
- Adding support for additional emergency classes (e.g., police vehicles) and multi-camera inputs.
- Incorporating edge AI accelerators like Google Coral or NVIDIA Jetson Nano to improve detection speed.

Overall, this system lays the groundwork for an intelligent, responsive, and scalable traffic control infrastructure capable of adapting to modern urban challenges while prioritizing public safety.

References

[1] H. Mojahidul, A. Kroeger, P. Kamelia, N. George, A. Be-Nazir, A. Kadir, and H.-J. Busch, "Emergency medical rescue services in dhaka city, bangladesh: A situational analysis and needs assessments," *International Journal of Critical Care and Emergency Medicine*, vol. 8, 03 2022.

[2] T. R. Tishi, "Study on frequency of fire incidents and fire fighting capacity in different land use categories of dhaka metropolitan area," 2015.

[3] R. Sinha, R. Pandey, and R. Pattnaik, "Deep learning for computer vision tasks: A review," 04 2018.

[4] M. Hussain, "Yolov5, yolov8 and yolov10: The go-to detectors for real-time vision.," 07 2024.

[5] Y. Zhang, Z. Guo, J. Wu, Y. Tian, H. Tang, and X. Guo, "Real-time vehicle detection based on improved yolo v5," *Sustainability*, vol. 14, p. 12274, 09 2022.

[6] H. Razalli and M. Alkawaz, "Emergency vehicle recognition and classification method using hsv color segmentation," pp. 284–289, 02 2020.

[7] P. Shinde, S. Yadav, S. Rudrake, and P. Kumbhar, "Smart traffic control system using yolo," *International Research Journal of Engineering and Technology*, vol. 6, pp. 169–172, 12 2019.

[8] G. Kavya and S. B, "Density based intelligent traffic signal system using pic microcontroller," *Engineering, Technology and Applied Science Research*, vol. 3, pp. 205–209, 01 2015.

[9] A. George, V. George, and M. George, "Iot based smart traffic light control system," pp. 148–151, 03 2018.

[10] G. Babu and G. Ashwin, *Intelligent Traffic Management System Using YOLO Machine Learning Model*, pp. 119–128. 01 2022.

[11] A. Navarro-Espinoza, O. R. López-Bonilla, E. E. García-Guerrero, E. Tlelo-Cuautle, D. López-Mancilla, C. Hernández-Mejía, and E. Inzunza-González, "Traffic flow prediction for smart traffic lights using machine learning algorithms," *Technologies*, vol. 10, no. 1, 2022.

[12] S. Roy and M. S. Rahman, "Emergency vehicle detection on heavy traffic road from cctv footage using deep convolutional neural network," in *2019 International Conference on Electrical, Computer and Communication Engineering (ECCE)*, pp. 1–6, 2019.

[13] B. C. C. Meng, N. S. Damanhuri, and N. A. Othman, "Smart traffic light control system using image processing," *IOP Conference Series: Materials Science and Engineering*, vol. 1088, p. 012021, 2021. Annual Conference on Computer Science and Engineering Technology (AC2SET) 2020.

[14] S. Geethapriya, N. Duraimurugan, and S. P. Chokkalingam, "Real-time object detection with yolo," *International Journal of Engineering and Advanced Technology (IJEAT)*, vol. 8, 02 2019. Available at: https://arxiv.org/pdf/2208.00773.

[15] A. Chockalingam, A. J. Antony Vincent, and A. S, "Clearing pathway for ambulance using yolo and raspberry pi.," 06 2021.

[16] A. Mecocci and C. Grassi, "Rtaiaed: A real-time ambulance in an emergency detector with a pyramidal part-based model composed of mfccs and yolov8," *Sensors*, vol. 24, no. 7, 2024.